COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 / Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	October 13, 2026	**DATE SUBMITTED:**	October 15, 2026
NAME:	Aguila, Daniel C. Torres, Carl Andrei D.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. HOA12.1 Algorithmic Strategies

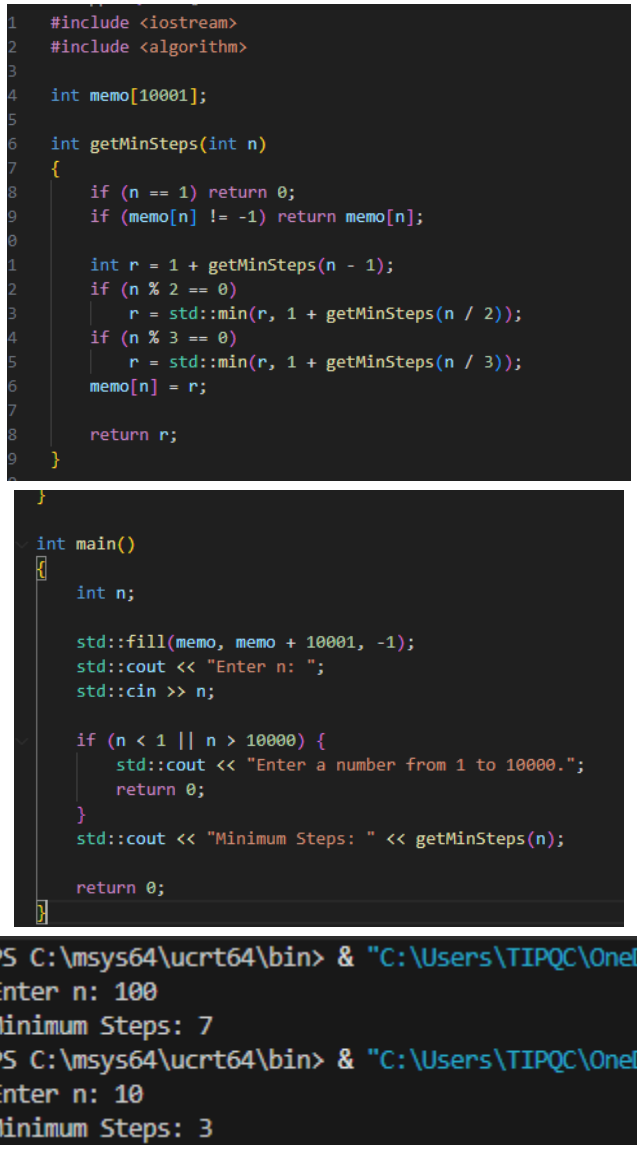
1. PROCEDURE OUTPUT

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

STRATEGY	ALGORITHM	ANALYSIS
Recursion	**Fibonacci Sequence**	The algorithm **calculates the value of the term using the function that calls itself with reduced values until base values are met**.
Brute Force	**Linear Search Algorithm**	**Linear Search Algorithm** has a **Brute Force Algorithm because it has a function that has a while loop where the loop consists of loops through the array by i<arrSize and checking if dataFind == data[i], which outputs that the user input (std::cin >> toFind) has been successfully been found and immediately exits the function via return**. The **brute force algorithm shows that if the loop continues without finding a match inside the randomly-generated array by the use of the cstdlib for the values of the array and syntax (int i = 0; i<max_size; i++) { dataset[i] = rand(); }, the**

		execution moves past the while loop and outputs that the user input element is not found, and returns the function which shows the brute force algorithm in this Linear Searching Algorithm.
Backtracking	**Depth-First Search (DFS)**	It **tries one direction as much as possible and retreats when it comes across an obstacle**. This way, it can reverse mistaken decisions and take other oaths.
Greedy	**Selection Sort**	The algorithm **continues to select the minimum item and puts it in place**. In this way, **it makes the best possible choice at each stage without changing any previous choices.**
Divide-and-Conquer	**Merge Sort**	**Merge Sort** has a **divide-and-conquer algorithm where the array is separated into subproblems until it separates into individual values**. **After comparing the two individual values in the subproblem**, it will be **merge and compare recursively by the use of a helper function that using a temporary array temp[right - left + 1] to combine them and a primary while loop that compares the values of my left sub-array at i and the right sub-array at j, and two cleanup loops that grab any elements remaining and merge them into one complete and organized array**.

Table 12-1. Algorithmic Strategies and Examples

Screenshot	
Analysis	This solution **makes use of recursion with memoization for solving the Minimum Steps to One problem, where the recurrence relation is F(n) = 1 + min{F(n-1), F(n/2), F(n/3)}**. The **solution will return 0 in the base case when `n == 1`**. The **solution tries all possible moves such as subtracting 1 always and dividing by 2 or 3 only when `n` is divisible by 2 or 3**. This is **done to overcome the limitation of the greedy algorithm in solving some cases, such as 10 where the greedy algorithm takes 4 moves, while this algorithm takes 3 moves**. The **solution utilizes an array called `memo` with all values equal to -1 to store all solved subproblems, so each value is computed once**. This **cuts down the computation time from exponential time to O(n)**.

```cpp
#include <iostream>
#include <algorithm>

int memo[10001];

int getMinSteps(int n)
{
    if (n == 1) return 0;
    if (memo[n] != -1) return memo[n];

    int r = 1 + getMinSteps(n - 1);
    if (n % 2 == 0)
        r = std::min(r, 1 + getMinSteps(n / 2));
    if (n % 3 == 0)
        r = std::min(r, 1 + getMinSteps(n / 3));
    memo[n] = r;

    return r;
}
```

```cpp
int main()
{
    int n;

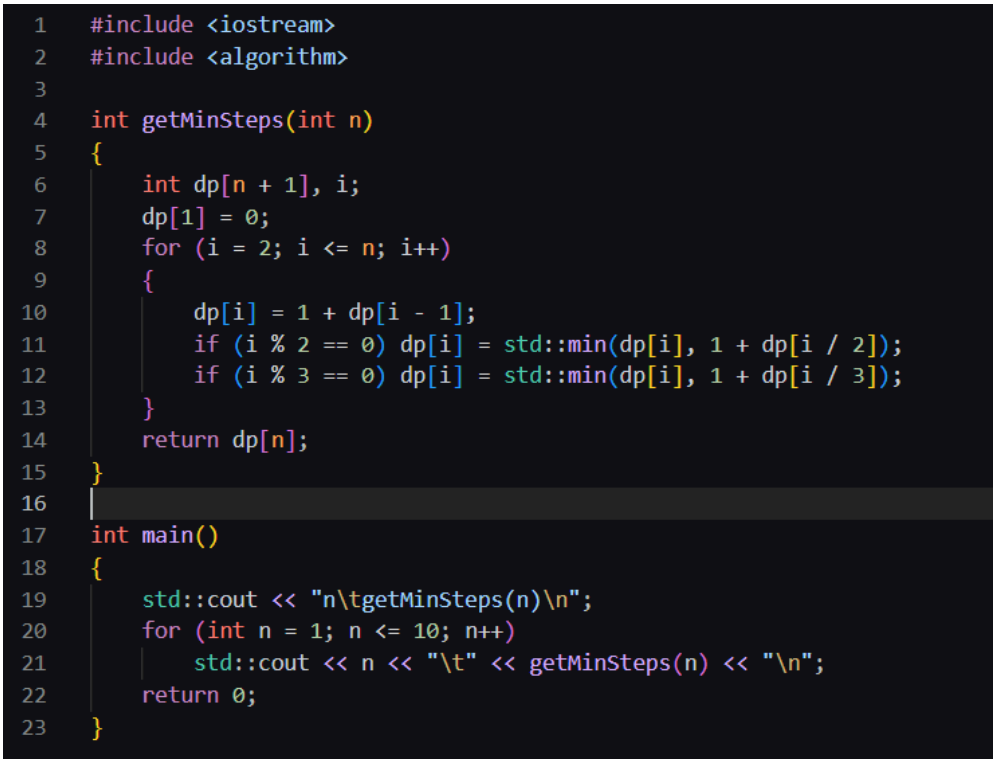
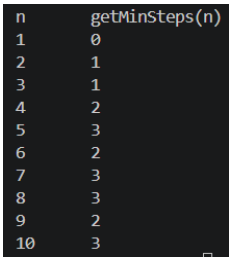
    std::fill(memo, memo + 10001, -1);
    std::cout << "Enter n: ";
    std::cin >> n;

    if (n < 1 || n > 10000) {
        std::cout << "Enter a number from 1 to 10000.";
        return 0;
    }
    std::cout << "Minimum Steps: " << getMinSteps(n);

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> & "C:\Users\TIPQC\OneDr
Enter n: 100
Minimum Steps: 7
PS C:\msys64\ucrt64\bin> & "C:\Users\TIPQC\OneDr
Enter n: 10
Minimum Steps: 3
```

Table 12-2. Memoization Implementation

Screenshot	 
Analysis	In this code, i've implemented the **bottom-up dynamic programming to find the minimum number of steps to reduce n to 1**, where **each of the step subtracts 1 or divides by 2 or 3**. I've set up the code **(dp[1] = 0) as the base case**, then for each i, I've started from **(1 + dp[i-1]) and used the min to keep the smaller value if the i is divisible by 2 or 3**. **I've implemented the main driver code to get the output console for (n = 1 to 10) is (0, 1, 1, 2, 3, 2, 3, 3, 2, 3)**, which **shows that the values don't always increase, since 6 needs only 2 steps while 5 needs 3**. This shows that the algorithm "Bottom-up dynamic programming has a time complexity of O(n) for best and worst case because of its addition, assignment, two modulo checks, and two min calls with array lookups which none of them depends on n (c * (n-1)) so the time complexity is still O(n).

```cpp
#include <iostream>
#include <algorithm>

int getMinSteps(int n)
{
    int dp[n + 1], i;
    dp[1] = 0;
    for (i = 2; i <= n; i++)
    {
        dp[i] = 1 + dp[i - 1];
        if (i % 2 == 0) dp[i] = std::min(dp[i], 1 + dp[i / 2]);
        if (i % 3 == 0) dp[i] = std::min(dp[i], 1 + dp[i / 3]);
    }
    return dp[n];
}

int main()
{
    std::cout << "n\tgetMinSteps(n)\n";
    for (int n = 1; n <= 10; n++)
        std::cout << n << "\t" << getMinSteps(n) << "\n";
    return 0;
}
```

```
n       getMinSteps(n)
1       0
2       1
3       1
4       2
5       3
6       2
7       3
8       3
9       2
10      3
```

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code + Output Console

```cpp
#include <iostream>
#include <vector>
#include <string>

int M, N;
std::vector<std::vector<int>> mat;
std::vector<std::vector<std::vector<int>>> memo;
int calls1 = 0, calls2 = 0;


//returns number of paths from (i, j) to the last cell with total cost = cost
int countPaths(int i, int j, int cost) {
    calls1++;

    if (i >= M || j >= N || cost < 0)
        return 0;

    if (i == M - 1 && j == N - 1) {
        if (cost == mat[i][j])
            return 1;
        return 0;
    }

    int down = countPaths(i + 1, j, cost - mat[i][j]);
    int right = countPaths(i, j + 1, cost - mat[i][j]);
    return down + right;
}

//this is recursion + memoization (so we don't solve the same subproblem again)
int countPathsMemo(int i, int j, int cost) {
    calls2++;
```

```cpp

//this is recursion + memoization (so we don't solve the same subproblem again)
int countPathsMemo(int i, int j, int cost) {
    calls2++;

    if (i >= M || j >= N || cost < 0)
        return 0;

    if (i == M - 1 && j == N - 1) {
        if (cost == mat[i][j])
            return 1;
        return 0;
    }

    if (memo[i][j][cost] != -1)
        return memo[i][j][cost];

    int down = countPathsMemo(i + 1, j, cost - mat[i][j]);
    int right = countPathsMemo(i, j + 1, cost - mat[i][j]);
    memo[i][j][cost] = down + right;
    return memo[i][j][cost];
}

//dp[i][j][c] = number of paths from (0,0) to (i,j) with total cost c
int countPathsDP(int K) {
    std::vector<std::vector<std::vector<int>>> dp(
        M, std::vector<std::vector<int>>(N, std::vector<int>(K + 1, 0)));

    if (mat[0][0] <= K)
        dp[0][0][mat[0][0]] = 1;
```

```
    if (mat[0][0] <= K)
        dp[0][0][mat[0][0]] = 1;

    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            for (int c = 0; c <= K; c++) {
                if (i == 0 && j == 0)
                    continue;
                if (c < mat[i][j])
                    continue;
                int before = c - mat[i][j];
                if (i > 0)
                    dp[i][j][c] += dp[i - 1][j][before];
                if (j > 0)
                    dp[i][j][c] += dp[i][j - 1][before];
            }
        }
    }
    return dp[M - 1][N - 1][K];
}

//print every path that has the given cost
void printPaths(int i, int j, int cost, std::string path) {
    if (i >= M || j >= N || cost < 0)
        return;

    path += std::to_string(mat[i][j]);

    if (i == M - 1 && j == N - 1) {
        if (cost == mat[i][j])
            std::cout << "  " << path << "\n";
        return;
    }
```

```
    path += " -> ";
    printPaths(i + 1, j, cost - mat[i][j], path);
    printPaths(i, j + 1, cost - mat[i][j], path);
}

int main() {
    int K;

    std::cout << "Enter number of rows and columns: ";
    std::cin >> M >> N;

    mat.assign(M, std::vector<int>(N));
    std::cout << "Enter the matrix:\n";
    for (int i = 0; i < M; i++)
        for (int j = 0; j < N; j++)
            std::cin >> mat[i][j];

    std::cout << "Enter the target cost: ";
    std::cin >> K;

    memo.assign(M, std::vector<std::vector<int>>(N, std::vector<int>(K + 1, -1)));

    int r1 = countPaths(0, 0, K);
    int r2 = countPathsMemo(0, 0, K);
    int r3 = countPathsDP(K);

    std::cout << "\nRecursion      : " << r1 << " path(s), " << calls1 << " calls\n";
    std::cout << "Memoization    : " << r2 << " path(s), " << calls2 << " calls\n";
    std::cout << "Bottom-up DP   : " << r3 << " path(s)\n";

    std::cout << "\nPaths with cost " << K << ":\n";
    printPaths(0, 0, K, "");

    return 0;
```

int calls1

Generate Copilot summa

```
Enter number of rows and columns: 4 4
Enter the matrix:
4 7 1 6
6 7 3 9
3 8 1 2
7 1 7 3
Enter the target cost: 25

Recursion     : 0 path(s), 77 calls
Memoization   : 0 path(s), 67 calls
Bottom-up DP  : 0 path(s)

Paths with cost 25:
```

Analysis	**Torres:**

Daniel's Conclusion/Learnings/Analysis	

Carl's Conclusion/Learnings/Analysis	

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.